Specyfikacja Architektury Systemu Przesyłek Kurierskich

Poniższa dokumentacja przedstawia strukturę klas, encji i modułów backendu (NestJS + TypeORM) dla systemu obsługi dostaw. Architektura opiera się na podziale na warstwę kontrolerów (obsługa zapytań HTTP/REST), warstwę serwisów (logika biznesowa) oraz warstwę encji bazodanowych, wspieranych przez bazę główną PostgreSQL (z rozszerzeniem PostGIS), bazę in-memory Redis oraz platformę Ably do komunikacji w czasie rzeczywistym.

1. Wykorzystany Stos Technologiczny (Tech Stack)

Architektura systemu została zaprojektowana w oparciu o jednolity ekosystem języka TypeScript (zarówno po stronie serwera, jak i aplikacji klienckich). Poniżej zestawiono główne technologie, bazy danych oraz usługi zewnętrzne wraz z uzasadnieniem ich wyboru.

1.1. Warstwa Serwerowa (Backend)

- **NestJS:** Główny framework backendowy. Wybrany ze względu na pełne wsparcie dla TypeScriptu oraz wbudowaną architekturę modularną (kontrolery, serwisy, dependency injection), która idealnie sprawdza się w implementacji złożonej logiki biznesowej, takiej jak automat stanów zamówień.
- **TypeORM:** System mapowania obiektowo-relacyjnego (ORM). Pozwala na wygodne odwzorowanie encji domenowych na tabele w bazie danych i posiada natywne wsparcie dla typów przestrzennych PostGIS, co znacząco ułatwia operacje na danych geograficznych.

1.2. Warstwa Składowania Danych (Bazy danych)

- **PostgreSQL z rozszerzeniem PostGIS:** Główna, relacyjna baza danych. PostgreSQL gwarantuje transakcyjność i spójność (ACID), co jest krytyczne przy obsłudze zamówień i płatności. Rozszerzenie PostGIS (indeksy GIST, funkcje takie jak `ST_DWithin`) zostało wybrane do wydajnego przeprowadzania zapytań przestrzennych, np. wyszukiwania dostawców w pobliżu klienta.
- **Redis:** Baza danych In-Memory (typu klucz-wartość). Wprowadzona jako warstwa odciążająca główną bazę relacyjną z ogromnej liczby operacji wejścia/wyjścia generowanych przez telemetrię. Używana do przechowywania lokalizacji kurierów na żywo (`GEOADD` , `GEOSEARCH`), obsługi mechanizmu wygasających ofert przydziału (klucze TTL) oraz asynchronicznych kolejek.

1.3. Warstwa Kliencka (Frontend)

- **React:** Framework do budowy aplikacji webowych typu Single Page Application (SPA). Zostanie wykorzystany do stworzenia Panelu Klienta (przeglądanie oferty, koszyk, śledzenie zamówień) oraz Panelu Dostawcy (zarządzanie menu, akceptacja nowych zleceń).
- **React Native (z Expo):** Framework mobilny pozwalający na kompilację kodu JS/TS - co w połączeniu z backendem TS się świetnie uzupełnia - do natywnych aplikacji na platformy iOS i Android. Został

wybrany do budowy aplikacji Kuriera, ponieważ jej kluczową funkcjonalnością jest ciągły dostęp do wbudowanego modułu GPS urządzenia (również w tle).

- **TanStack Query (React Query) i Zustand:** Główne biblioteki zarządzania stanem na frontendzie. React Query odpowiada za płynną synchronizację danych z serwerem REST (cache'owanie zapytań), a Zustand za lekki stan lokalny (np. obsługę koszyka zakupowego przed złożeniem zamówienia).

1.4. Usługi Zewnętrzne i Integracje

- **Ably:** Platforma chmurowa Real-Time (wzorzec Pub/Sub). Przejęła odpowiedzialność za utrzymanie otwartych połączeń z aplikacjami frontendowymi. Została wybrana w celu odciążenia serwera NestJS ze skomplikowanego zarządzania stanem websocketów, zapewniając niezawodne powiadomienia oraz bieżące aktualizacje stanu przesyłki w panelach frontendowych.
- **Stripe:** Zewnętrzna bramka płatnicza. Wybrana ze względu na bogate API, wbudowane bezpieczeństwo transakcji oraz niezawodny system webhooków asynchronicznie informujący backend o pomyślnym zaksięgowaniu wpłaty.
- **Nominatim (OpenStreetMap API):** Zewnętrzny serwis geokodujący. Wybrany jako darmowa alternatywa dla płatnych API mapowych do bezproblemowej konwersji adresów tekstowych (wprowadzanych przez klienta) na współrzędne geograficzne niezbędne dla systemu geolokalizacji.

1.5. Infrastruktura i Wdrożenie

Zdecydowaliśmy się na oparcie wdrożenia systemu na zarządzanych usługach chmurowych z ekosystemu Google Cloud.

- **Google Cloud Run (Backend):** Środowisko uruchomieniowe serverless dla kontenerów Docker z aplikacją NestJS. Zapewnia automatyczne skalowanie od zera do tysięcy instancji w zależności od aktualnego obciążenia ruchem sieciowym, optymalizując koszty infrastruktury.
- **Cloud SQL for PostgreSQL (Baza Relacyjna):** Zarządzana usługa bazy danych. Zapewnia wysoką dostępność (HA), zautomatyzowane kopie zapasowe oraz natywne wsparcie dla rozszerzenia **PostGIS**, gwarantując stabilność kluczowych danych biznesowych.
- **Cloud Memorystore for Redis (Baza In-memory):** Zarządzany klaster Redis zintegrowany w prywatnej sieci chmurowej (VPC). Gwarantuje mikrosekundowe czasy odpowiedzi niezbędne dla cyklicznej telemetrii kurierów, chroniąc system przed utratą danych.
- **Firebase Hosting (Frontend Web):** Prosty i bezpieczny hosting statycznych plików aplikacji React. Wykorzystuje CDN Google, co skraca czas ładowania interfejsów.
- **Expo EAS:** Platforma chmurowa wykorzystywana do budowania gotowych pakietów instalacyjnych aplikacji mobilnej Kuriera (`.apk` / `.ipa`), co wyeliminuje konieczność konfiguracji lokalnych środowisk kompilacji.
- **Github:** Platforma do kontroli wersji i dzielenia pracy między deweloperami.

---

2. Moduły Główne (Kontrolery i Serwisy)

Moduł Autoryzacji (Auth)

Odpowiada za rejestrację ról, logowanie i wydawanie tokenów JWT.

`AuthController`

- `register(dto: RegisterDto) -> Promise<AuthResponseDto>`: Rejestruje użytkownika i tworzy profil (zapis transakcyjny).
- `login(dto: LoginDto) -> Promise<AuthResponseDto>`: Weryfikuje dane logowania i zwraca tokeny (Access i Refresh).
- `refresh(dto: RefreshTokenDto) -> Promise<AuthTokensDto>`: Odświeża wygasły access token.

`AuthService`

- `registerUser(dto: RegisterDto) -> Promise<AuthResponseDto>`: Hashuje hasło i w razie potrzeby korzysta z geocodingu dla lokalizacji dostawcy.
- `validateUser(email: string, pass: string) -> Promise<UserEntity>`: Pobiera i weryfikuje użytkownika.
- `generateTokens(user: UserEntity) -> Promise<AuthTokensDto>`: Generuje nowe tokeny JWT z rolą (`role`) i identyfikatorem (`sub: user.id`) w payloadzie.

Moduł Użytkowników (Users)

Zarządza danymi profilowymi i statusem kuriera.

`UsersController`

- `getMyProfile(currentUser: JwtPayload) -> Promise<UserProfileDto>`: Zwraca dane konta wraz z profilem VENDOR/COURIER.
- `updateProfile(currentUser: JwtPayload, dto: UpdateProfileDto) -> Promise<UserProfileDto>`: Aktualizuje m.in. numer telefonu, imię i nazwisko.
- `toggleCourierAvailability(currentUser: JwtPayload, dto: AvailabilityDto) -> Promise<CourierStatusDto>`: Aktywuje/dezaktywuje kuriera w systemie.

`UsersService`

- `findByIdWithProfile(userId: number) -> Promise<UserEntity>`: Pobiera użytkownika łącząc bazowe tabele (JOIN).
- `setCourierAvailability(courierId: number, isAvailable: boolean, initialLocation?: PointDto) -> Promise<void>`: Ustawia flagę w bazie PostgreSQL oraz w razie wyłączenia – usuwa z puli wolnych kurierów w Redisie.

Moduł Katalogu (Products)

Obsługuje menu/ofertę dostawców.

`ProductsController`

- `getVendorsCatalog(query: SearchVendorsQueryDto) -> Promise<VendorSummaryDto[]>`: Wyszukiwanie dostawców, opcjonalnie uwzględniając odległość.
- `getVendorProducts(vendorId: number, includeInactive: boolean) -> Promise<ProductDto[]>`: Zwraca menu dostawcy.
- `createProduct(currentUser: JwtPayload, dto: CreateProductDto) -> Promise<ProductDto>`: Dodaje nowy produkt do oferty.
- `updateProduct(currentUser: JwtPayload, productId: number, dto: UpdateProductDto) -> Promise<ProductDto>`: Edytuje lub deaktywuje produkt (bez usuwania z bazy, by chronić historię).

`ProductsService`

- `create(vendorId: number, dto: CreateProductDto) -> Promise<ProductEntity>`: Dodaje nowy produkt.
- `update(vendorId: number, productId: number, dto: UpdateProductDto) -> Promise<ProductEntity>`: Uaktualnia stan produktu.
- `validateAndLockProductsForOrder(vendorId: number, items: OrderItemInputDto[]) -> Promise<ValidatedOrderItem[]>`: Weryfikuje aktywność produktów i pobiera ich aktualną cenę na moment składania zamówienia.

Moduł Zamówień (Orders)

Centralny automat stanów (State Machine) orkiestrujący cykl życia zlecenia.

`OrdersController`

- `createOrder(currentUser: JwtPayload, dto: CreateOrderDto) -> Promise<OrderCreatedResponseDto>`: Inicjuje proces, waliduje koszyk i zwraca URL do opłacenia.
- `getOrders(currentUser: JwtPayload, filter: OrderFilterDto) -> Promise<OrderDetailsDto[]>`: Zwraca listę zamówień per aktor.
- `getOrderById(currentUser: JwtPayload, orderId: number) -> Promise<OrderDetailsDto>`: Zwraca pełne szczegóły zamówienia (wymaga uprawnień udziałowca).
- `cancelOrderByClient(currentUser: JwtPayload, orderId: number) -> Promise<OrderDetailsDto>`: Stan `ORDERED` -> `CANCELLED`.
- `vendorAcceptOrder(currentUser: JwtPayload, orderId: number) -> Promise<OrderDetailsDto>`: Stan `ORDERED` -> `ACCEPTED`. Wyzwala szukanie kuriera.
- `vendorRejectOrder(currentUser: JwtPayload, orderId: number, dto: RejectReasonDto) -> Promise<OrderDetailsDto>`: Odrzucenie zamówienia (`REJECTED`) zlecające zwrot środków

(Refund).

- `vendorStartPreparation(currentUser: JwtPayload, orderId: number) -> Promise<OrderDetailsDto>`: Stan `ACCEPTED` -> `PREPARATION`.
- `courierRespondToOffer(currentUser: JwtPayload, orderId: number, dto: CourierOfferResponseDto) -> Promise<void>`: Akceptacja/odrzucenie oferty w ramach 15s soft-locka z Redis.
- `courierPickupOrder(currentUser: JwtPayload, orderId: number) -> Promise<OrderDetailsDto>`: Stan `PREPARATION` -> `SHIPPING`.
- `courierDeliverOrder(currentUser: JwtPayload, orderId: number) -> Promise<OrderDetailsDto>`: Stan `SHIPPING` -> `DELIVERED`.
- `courierReportIssue(currentUser: JwtPayload, orderId: number, dto: IssueReportDto) -> Promise<void>`: Obsługa wyjątków (awaria/wypadek) – ponowne szukanie kuriera lub przejście w stan `DELIVERY_FAILED`.

`OrdersService`

- `create(clientId: number, dto: CreateOrderDto) -> Promise<{ order: OrderEntity, paymentUrl: string }>`: Inicjalizuje nowe zamowienie.
- `transitionState(orderId: number, actorId: number, actorRole: UserRole, targetState: OrderStatus) -> Promise<OrderEntity>`: Sprawdza macierz przejść, zapisuje stan i triggeruje powiadomienia (Ably + DB).
- `initiateCourierMatching(orderId: number) -> Promise<void>`: Odpytuje Redis o kuriera i wysyła ofertę.
- `confirmCourierAssignment(orderId: number, courierId: number) -> Promise<void>`: Twardy zapis w PostgreSQL, usuwa wolnego kuriera z Redis.

Moduł Geolokalizacji (Geolocation)

Integracja zewnętrzna (geokodowanie) i przestrzenna baza Redis (telemetria).

`GeolocationController`

- `pingLocation(currentUser: JwtPayload, dto: LocationPingDto) -> Promise<void>`: Odbiera ping GPS od wolnego kuriera (co 10s).
- `geocodePreview(query: AddressQueryDto) -> Promise<GeocodedAddressDto>`: Waliduje wpisany adres dla frontendu.

`GeolocationService`

- `geocodeAddress(address: string) -> Promise<PointDto>`
- `updateCourierLocationInRedis(courierId: number, point: PointDto) -> Promise<void>`: Komenda `GEOADD` + odświeżenie TTL `active`.

- `findAndLockNearestCourier(vendorLocation, orderId, excludedCouriers) -> Promise<number | null>`: Używa `GEOSEARCH` i skryptu LUA do założenia 15-sekundowego soft-locka na kandydata.
- `releaseCourierSoftLock(courierId: number, orderId: number) -> Promise<void>`: Zwalnai soft-lock.
- `removeCourierFromActivePool(courierId: number) -> Promise<void>`
- `findNearbyVendors(clientPoint: PointDto, radiusMeters: number) -> Promise<VendorDistanceResult[]>`: Używa `ST_DWithin` oraz indeksu GIST `idx_vendor_location` w bazie PostGIS.

Moduł Płatności (Payments)

`PaymentsController`

- `handleWebhook(signature: string, payload: PaymentWebhookPayloadDto) -> Promise<void>`: Asynchroniczne webhooki od operatora.
- `getPaymentStatus(currentUser: JwtPayload, orderId: number) -> Promise<PaymentDto>`

`PaymentsService`

- `createPaymentIntent(orderId: number, amount: number, clientEmail: string) -> Promise<string>`: Inicjalizuje płatność.
- `processWebhookEvent(payload: PaymentWebhookPayloadDto) -> Promise<void>`: Aktualizuje status, a jeśli `COMPLETED` – informuje Dostawcę.
- `refundPayment(orderId: number) -> Promise<void>`: Zleca zwrot na kartę klienta.

Moduł Ocen (Rating)

`RatingController`

- `createRating(currentUser: JwtPayload, orderId: number, dto: CreateRatingDto) -> Promise<RatingDto>`
- `getTargetRatings(targetUserId: number, query: PaginationDto) -> Promise<RatingListDto>`

`RatingService`

- `submitRating(clientId, orderId, dto) -> Promise<RatingEntity>`: Waliduje stan `DELIVERED`, pilnuje więzów unikalności (`uq_order_target_rating`) i przelicza średnią ocen (`AVG`) użytkownika docelowego.

Moduł Powiadomień (Notifications)

Integracja Ably (wzorzec Pub/Sub) oraz trwała historia komunikatów.

`NotificationController`

- `getAblyToken(currentUser: JwtPayload) -> Promise<AblyTokenRequestDto>`: Podpisuje TokenRequest (zależny od roli) pozwalający klientowi łączyć się bezpiecznie z siecią Ably.
- `getMyNotifications(currentUser: JwtPayload, query: PaginationDto) -> Promise<NotificationDto[]>`: Historia alertów z bazy (do wglądu offline).
- `markAsRead(currentUser: JwtPayload, notificationId: number) -> Promise<void>`

`NotificationService`

- `createAblyTokenRequest(userId: number, role: UserRole) -> Promise<AblyTokenRequestDto>`: Pobiera token Ably dla użytkownika.
- `publishAndPersistMessage(...) -> Promise<void>`: Zapisuje rekord do `notifications` i wysyła asynchroniczny POST do API Ably.
- `sendCourierOfferAlert(...) -> Promise<void>`: Lotny komunikat dla kuriera (przydział), nietrafiający trwale do bazy.

---

3. Schematy Encji Bazodanowych (TypeORM)

Poniższe klasy reprezentują odwzorowanie relacyjnego schematu `schema.sql` na obiekty domenowe TypeScript (TypeORM). Współrzędne geograficzne `geography(Point, 4326)` są mapowane w kodzie na standardowy interfejs GeoJSON `Point` (`{ type: 'Point', coordinates: [longitude, latitude] }`).

Typy wyliczeniowe (Enums)

- **`UserRole`**: `'CLIENT' | 'VENDOR' | 'COURIER'`
- **`OrderStatus`**: `'ORDERED' | 'ACCEPTED' | 'PREPARATION' | 'SHIPPING' | 'DELIVERED' | 'REJECTED' | 'CANCELLED' | 'DELIVERY_FAILED'`
- **`PaymentStatus`**: `'PENDING' | 'COMPLETED' | 'FAILED' | 'REFUNDED'` *(Uwaga: warto dopisać `'REFUNDED'` do więzów `CHECK` w tabeli `payments` w `schema.sql`)*
- **`TargetRole`**: `'VENDOR' | 'COURIER'`

3.1. `UserEntity` (Tabela: `users`)

Główna encja reprezentująca konto dowolnego użytkownika w systemie.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `email: string` – Unikalny adres e-mail (`varchar(100)`, `unique`, `not null`).

- `passwordHash: string` – Hash hasła (`password_hash varchar(255)` , `not null` , kolumna z flagą `select: false` w ORM).
- `role: UserRole` – Rola w systemie (`varchar(20)` , `not null`).
- `firstName: string` – Imię (`first_name varchar(50)` , `not null`).
- `lastName: string` – Nazwisko (`last_name varchar(50)` , `not null`).
- `phone: string` – Numer telefonu (`varchar(20)` , `not null`).
- `createdAt: Date` – Data utworzenia (`created_at timestamp` , `default current_timestamp`).

- **Relacje:**
  - `vendorProfile?: VendorProfileEntity` – `@OneToOne(() => VendorProfileEntity, vp => vp.user)`
  - `courierProfile?: CourierProfileEntity` – `@OneToOne(() => CourierProfileEntity, cp => cp.user)`
  - `clientOrders?: OrderEntity[]` – `@OneToMany(() => OrderEntity, o => o.client)`
  - `notifications?: NotificationEntity[]` – `@OneToMany(() => NotificationEntity, n => n.user)`

3.2. `VendorProfileEntity` (Tabela: `vendor_profiles`)

Profil rozszerzający konto użytkownika o roli `VENDOR` (współdzielony klucz główny 1:1). Posiada indeks przestrzenny GIST `idx_vendor_location` .

- **Pola:**
  - `userId: number` – Klucz główny i obcy (`user_id integer primary key`).
  - `companyName: string` – Nazwa lokalu/firmy (`company_name varchar(100)` , `not null`).
  - `location: Point` – Lokalizacja geograficzna lokalu (`geography(Point, 4326)` , `not null`).
  - `avgRating: number` – Średnia ocen (`avg_rating decimal(3,2)` , `default 0.00`).
- **Relacje:**
  - `user: UserEntity` – `@OneToOne(() => UserEntity, { onDelete: 'CASCADE' })` , `@JoinColumn({ name: 'user_id' })`
  - `products: ProductEntity[]` – `@OneToMany(() => ProductEntity, p => p.vendor)`
  - `orders: OrderEntity[]` – `@OneToMany(() => OrderEntity, o => o.vendor)`

3.3. `CourierProfileEntity` (Tabela: `courier_profiles`)

Profil rozszerzający konto użytkownika o roli `COURIER` (współdzielony klucz główny 1:1). Pozycja GPS na żywo jest przechowywana w bazie Redis, natomiast tabela przechowuje trwały stan konta.

- **Pola:**

- `userId: number` – Klucz główny i obcy (`user_id integer primary key`).
- `isAvailable: boolean` – Czy kurier jest gotowy do pracy (`is_available boolean`, `default false`).
- `avgRating: number` – Średnia ocen (`avg_rating decimal(3,2)`, `default 0.00`).

- **Relacje:**
  - `user: UserEntity` – `@OneToOne(() => UserEntity, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'user_id' })`
  - `orders: OrderEntity[]` – `@OneToMany(() => OrderEntity, o => o.courier)`

3.4. `ProductEntity` (Tabela: `products`)

Pozycja w katalogu (menu) danego dostawcy.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `vendorId: number` – ID dostawcy (`vendor_id integer`, `not null`).
  - `name: string` – Nazwa produktu (`varchar(100)`, `not null`).
  - `price: number` – Aktualna cena jednostkowa (`decimal(10,2)`, `check (price >= 0)`).
  - `isActive: boolean` – Flaga widoczności w ofercie (`is_active boolean`, `default true`).

- **Relacje:**
  - `vendor: VendorProfileEntity` – `@ManyToOne(() => VendorProfileEntity, vp => vp.products, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'vendor_id' })`
  - `orderItems: OrderItemEntity[]` – `@OneToMany(() => OrderItemEntity, oi => oi.product)`

3.5. `OrderEntity` (Tabela: `orders`)

Centralna encja przechowująca stan zamówienia oraz dane adresowe dostawy.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `clientId: number` – ID klienta (`client_id integer`, `not null`).
  - `vendorId: number` – ID dostawcy (`vendor_id integer`, `not null`).
  - `courierId: number | null` – ID przypisanego kuriera (`courier_id integer`, `nullable`).
  - `status: OrderStatus` – Stan zamówienia (`order_status enum`, `default 'ORDERED'`).
  - `deliveryLocation: Point` – Punkt docelowy GPS (`delivery_location geography(Point, 4326)`, `not null`).
  - `deliveryAddress: string` – Tekstowy adres dostawy (`delivery_address varchar(100)`, `not null`).

- `totalPrice: number` – Łączna kwota zamówienia (`total_price decimal(10,2)`, `check (total_price >= 0)`).
- `createdAt: Date` – Czas złożenia zamówienia (`created_at timestamp`, `default current_timestamp`).

- **Relacje:**
  - `client: UserEntity` – `@ManyToOne(() => UserEntity, { onDelete: 'RESTRICT' })`, `@JoinColumn({ name: 'client_id' })`
  - `vendor: VendorProfileEntity` – `@ManyToOne(() => VendorProfileEntity, { onDelete: 'RESTRICT' })`, `@JoinColumn({ name: 'vendor_id' })`
  - `courier: CourierProfileEntity | null` – `@ManyToOne(() => CourierProfileEntity, { nullable: true, onDelete: 'SET NULL' })`, `@JoinColumn({ name: 'courier_id' })`
  - `items: OrderItemEntity[]` – `@OneToMany(() => OrderItemEntity, oi => oi.order, { cascade: true })`
  - `payment: PaymentEntity` – `@OneToOne(() => PaymentEntity, p => p.order)`
  - `ratings: RatingEntity[]` – `@OneToMany(() => RatingEntity, r => r.order)`
  - `notifications: NotificationEntity[]` – `@OneToMany(() => NotificationEntity, n => n.order)`

3.6. `OrderItemEntity` (Tabela: `order_items`)

Pozycja zamówienia zamrażająca cenę produktu w momencie zakupu.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `orderId: number` – ID zamówienia (`order_id integer`, `not null`).
  - `productId: number` – ID produktu (`product_id integer`, `not null`).
  - `quantity: number` – Liczba sztuk (`integer`, `check (quantity > 0)`).
  - `unitPrice: number` – Zamrożona cena jednostkowa (`unit_price decimal(10,2)`, `check (unit_price >= 0)`).

- **Relacje:**
  - `order: OrderEntity` – `@ManyToOne(() => OrderEntity, o => o.items, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'order_id' })`
  - `product: ProductEntity` – `@ManyToOne(() => ProductEntity, { onDelete: 'RESTRICT' })`, `@JoinColumn({ name: 'product_id' })`

3.7. `PaymentEntity` (Tabela: `payments`)

Rejestr transakcji płatniczej powiązanej 1:1 z zamówieniem.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `orderId: number` – Unikalny klucz obcy zamówienia (`order_id integer`, `unique`, `not null`).
  - `amount: number` – Kwota transakcji (`decimal(10,2)`, `check (amount >= 0)`).
  - `status: PaymentStatus` – Status płatności (`varchar(20)`, `default 'PENDING'`).
  - `externalTxId: string | null` – Identyfikator z zewnętrznej bramki płatniczej (`external_tx_id varchar(100)`, `nullable`).
  - `updatedAt: Date` – Czas ostatniej zmiany statusu (`updated_at timestamp`, `default current_timestamp`).
- **Relacje:**
  - `order: OrderEntity` – `@OneToOne(() => OrderEntity, o => o.payment, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'order_id' })`

3.8. `RatingEntity` (Tabela: `ratings`)

Ocena i komentarz wystawiony przez klienta dla dostawcy lub kuriera. Posiada ograniczenie unikalności `UNIQUE (order_id, target_role)`.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `orderId: number` – ID ocenianego zamówienia (`order_id integer`, `not null`).
  - `clientId: number` – ID wystawiającego klienta (`client_id integer`, `not null`).
  - `targetUserId: number` – ID ocenianego użytkownika (`target_user_id integer`, `not null`).
  - `targetRole: TargetRole` – Oceniana rola (`target_role varchar(20)`, `'VENDOR' | 'COURIER'`).
  - `score: number` – Ocena w skali 1–5 (`integer`, `check (score between 1 and 5)`).
  - `comment: string` – Treść komentarza (`varchar(50)`, `not null` – *zalecana zmiana w DDL na `varchar(500)`*).
  - `createdAt: Date` – Data wystawienia (`created_at timestamp`, `default current_timestamp`).
- **Relacje:**
  - `order: OrderEntity` – `@ManyToOne(() => OrderEntity, o => o.ratings, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'order_id' })`
  - `client: UserEntity` – `@ManyToOne(() => UserEntity, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'client_id' })`

- `targetUser: UserEntity` – `@ManyToOne(() => UserEntity, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'target_user_id' })`

3.9. `NotificationEntity` (Tabela: `notifications`)

Trwały zapis powiadomienia o zmianie statusu zamówienia, pozwalający na odczyt historii po zalogowaniu.

- **Pola:**
  - `id: number` – Klucz główny (`serial primary key`).
  - `userId: number` – ID adresata powiadomienia (`user_id integer`, `not null`).
  - `orderId: number` – ID powiązanego zamówienia (`order_id integer`, `not null`).
  - `message: string` – Treść komunikatu (`varchar(255)`, `not null`).
  - `isRead: boolean` – Czy odczytano (`is_read boolean`, `default false`).
  - `createdAt: Date` – Czas wygenerowania (`created_at timestamp`, `default current_timestamp`).
- **Relacje:**
  - `user: UserEntity` – `@ManyToOne(() => UserEntity, u => u.notifications, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'user_id' })`
  - `order: OrderEntity` – `@ManyToOne(() => OrderEntity, o => o.notifications, { onDelete: 'CASCADE' })`, `@JoinColumn({ name: 'order_id' })`

---

4. Modele Danych Wejściowych (Request DTOs)

Obiekty te wykorzystują `class-validator` dla zachowania bezpieczeństwa i rygoru danych na wejściu API.

- **Auth & Użytkownicy:**
  - `RegisterDto`: `email`, `password`, `role`, `firstName`, `lastName`, `phone`, opcjonalnie: `companyName`, `address`.
  - `LoginDto`: `email`, `password`.
  - `RefreshTokenDto`: `refreshToken`.
  - `UpdateProfileDto`: Opcjonalne pola (imiona, telefon, firma).
  - `AvailabilityDto`: `isAvailable`, `currentLocation` (PointDto).
- **Produkty:**
  - `CreateProductDto` / `UpdateProductDto`: `name`, `price`, `isActive`.
- **Zamówienia:**
  - `CreateOrderDto`: `vendorId`, `deliveryAddress`, `items` (tablica obiektów `productId` i `quantity`).

    - `CourierOfferResponseDto`: `accepted` (boolean).
    - `RejectReasonDto`: `reason` (string).
    - `IssueReportDto`: `type` (enum awarii), `description`.

- **Geolokalizacja:**
    - `PointDto`: `longitude`, `latitude`.
    - `LocationPingDto`: Rozszerza PointDto.
    - `AddressQueryDto`: `address` (string wyszukiwania).
    - `SearchVendorsQueryDto`: `longitude`, `latitude`, `radiusKm`.

- **Webhooki:**
    - `PaymentWebhookPayloadDto`: `externalTxId`, `status`, `amount`.

- **Oceny:**
    - `CreateRatingDto`: `targetRole` (VENDOR / COURIER), `score` (1-5), `comment` (do 50 znaków wg obecnego DDL, lub do 500 po korekcie).

- **Filtry / Inne:**
    - `PaginationDto`: `limit`, `offset`.
    - `OrderFilterDto`: `status`, `fromDate`, `toDate`.

---

5. Middleware & Guardy (Zabezpieczenia HTTP)

Klasy i dekoratory chroniące aplikację przed nieuprawnionym dostępem:

1. **`JwtAuthGuard`:** Weryfikuje strukturę i ważność nagłówka `Authorization: Bearer <token>`. Dołącza obiekt użytkownika zdekodowany z tokenu (`sub`, `role`) do obiektu Request jako argument dla `@CurrentUser()`.
2. **`RolesGuard`:** Wymaga dodania dekoratora `@Roles('CLIENT' | 'VENDOR' | 'COURIER')` na endpoincie. Odrzuca żądania z kodem 403 Forbidden, jeśli aktor wykonuje zapytanie niezgodne z jego interfejsem (np. klient nie może tworzyć produktów).
3. **`OrderParticipantGuard`:** Ochrona typu IDOR (Insecure Direct Object Reference). Wykorzystuje parametr URL `orderId`, by sprawdzić, czy zalogowany użytkownik widnieje w bazie jako `client_id`, `vendor_id`, `courier_id` zamówienia (lub ma założony soft-lock w Redisie przy przydziale).
4. **`WebhookSignatureGuard`:** Weryfikuje kryptograficzny Hash z użyciem `WEBHOOK_SECRET`, aby uniemożliwić atakującym ręczne wywoływanie webhooka ze sfabrykowanym statusem płatności `COMPLETED`.

6. Rozwiązanie najbardziej problematycznego obszaru

W celu odciążenia bazy relacyjnej z operacji owysokiej częstotliwości zapisu i odczytu, zdecydowaliśmy się wykorzystać bazę danych in-memory - Redis. Wykorzystujemy go przede wszystkim do śledzenia

geolokalizacji kurierów w czasie rzeczywistym oraz zarządzania krótkotrwałymi sesjami i rozproszonymi blokadami (ochrona przed race conditions).

6.1. Paradygmat: Redis vs PostgreSQL (SQL)

- **Struktura Klucz-Wartość:** Całą bazę traktujemy jak globalny zbiór zmiennych (słownik / hashmapę). Aby odczytać dane, nie wykonujemy zapytania `SELECT * FROM ... WHERE id = ...`, lecz pobieramy wartość przypisaną do konkretnego, unikalnego klucza (np. `courier:session:42`).
- **Trwałość vs Szybkość:** PostgreSQL zapisuje dane na dysku (trwałość transakcyjna ACID), natomiast Redis operuje w pamięci RAM. Odczyt i zapis trwają ułamki milisekund, co czyni go idealnym buforem dla danych ulotnych (np. bieżąca pozycja GPS), których ewentualna utrata nie zaburzy krytycznych procesów biznesowych.
- **Czas życia danych (TTL):** W SQL dane istnieją do momentu wywołania komendy `DELETE`. W Redisie niemal każdy klucz otrzymuje parametr TTL (Time-To-Live). Po upływie tego czasu serwer automatycznie usuwa dany klucz z pamięci.

6.2 Opis struktur w projekcie

A. couriers:locations (Struktura GEO)

Aplikacja mobilna wysyła metadane GPS bezpośrednio do Redisa. Struktura `GEO` zachowuje się pod maską jak posortowany zbiór (Sorted Set) i korzysta ze zoptymalizowanych algorytmów przestrzennych.

- **Wykorzystanie:** Gdy zamówienie przechodzi w stan "ACCEPTED", moduł przydziału wywołuje jedną komendę (np. `GEOSEARCH couriers:locations BYRADIUS 5 km`). Baza w czasie poniżej milisekundy zwraca listę ID najbliższych kurierów, całkowicie omijając obciążające zapytania przestrzenne w PostGIS.
- **Aktualizacja:** Wykonywana synchronicznie wraz z zapisem sesji kuriera (przy każdym pingu z aplikacji mobilnej).

B. courier:session:[courier_id] (Struktura HASH)

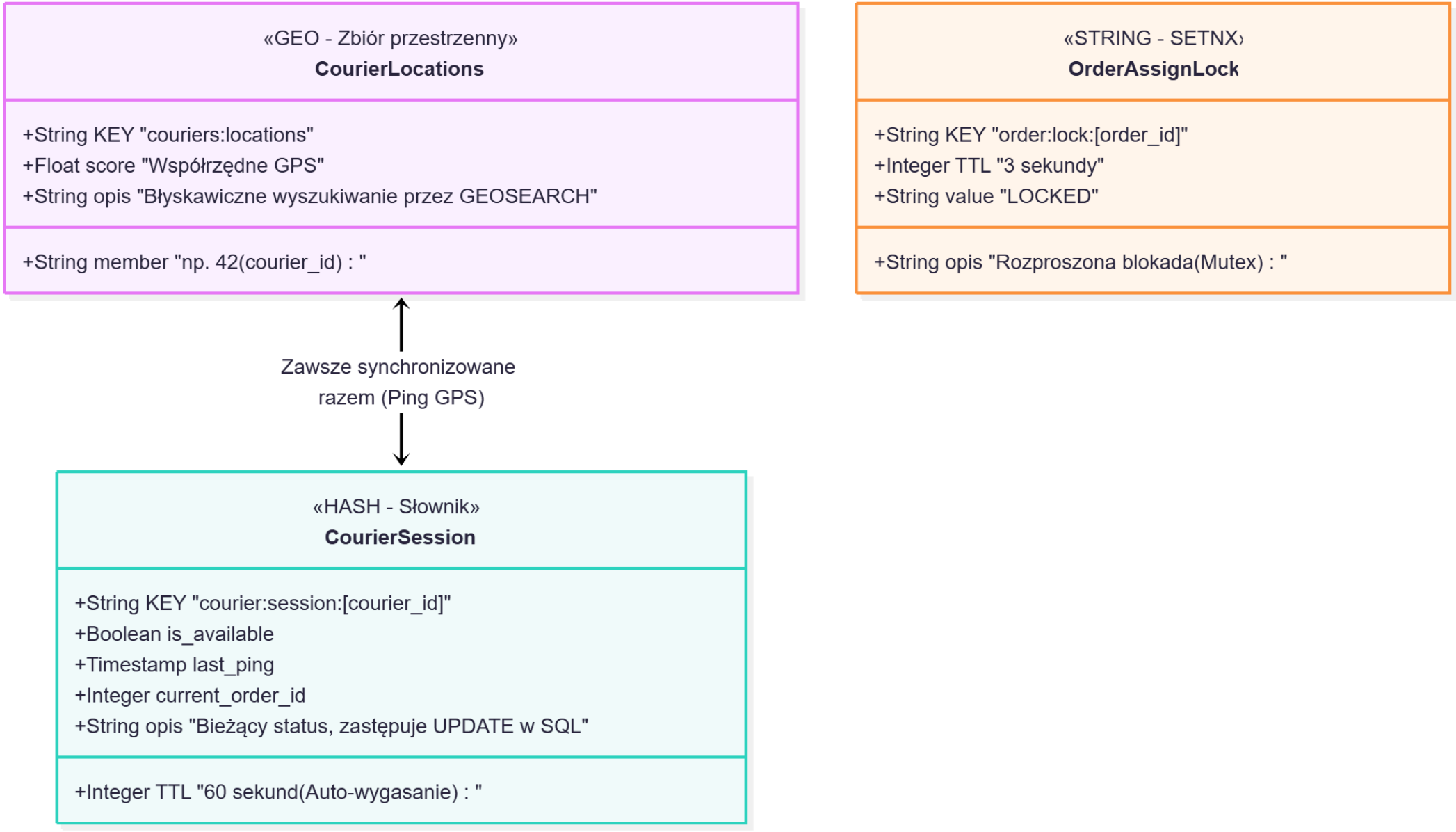
Słownik przechowujący bieżący stan kuriera. Pozwala bez obciążenia bazy SQL wysyłać i odbierać zapytania o to, czy kurier jest zalogowany i wolny.

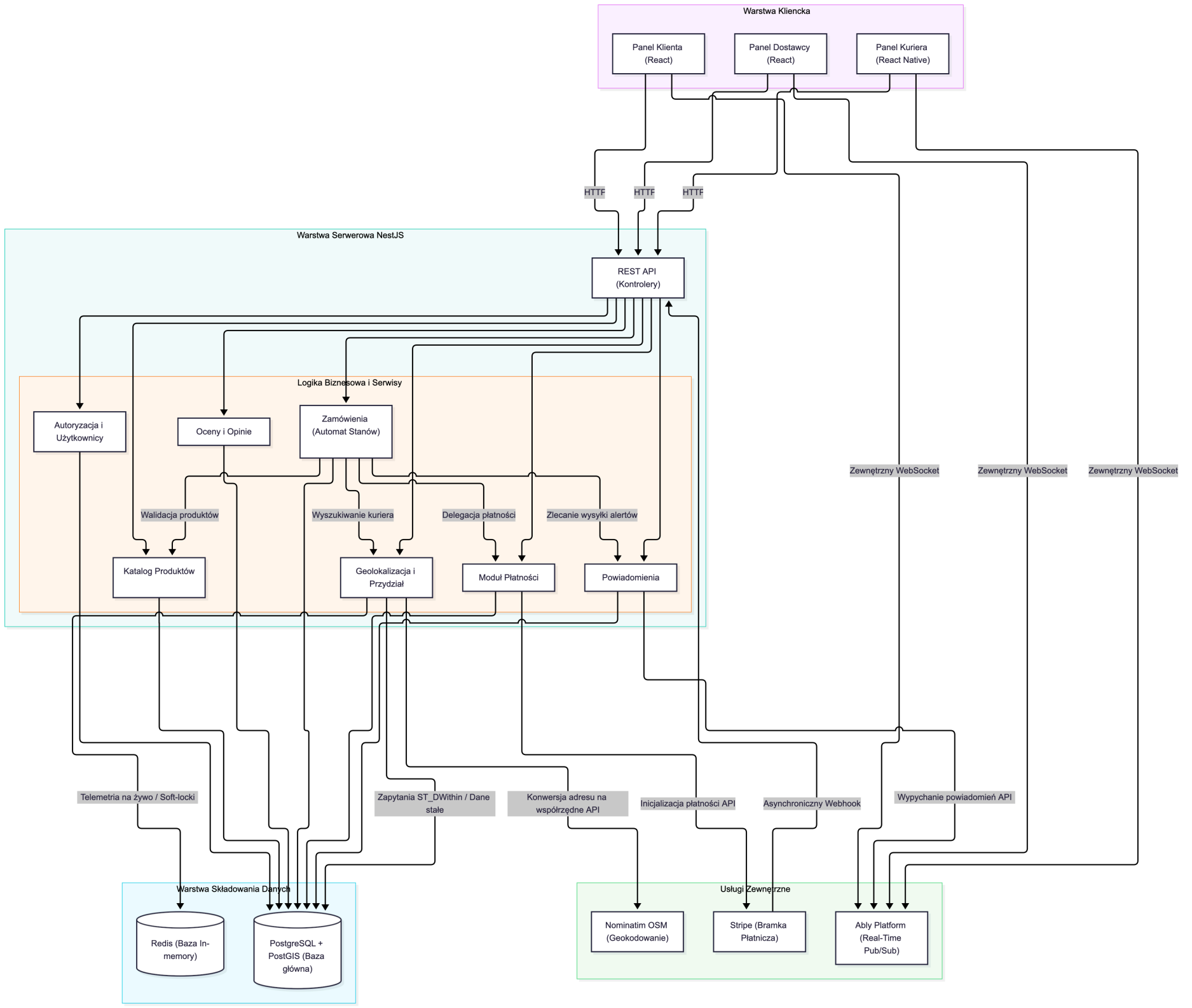
- **Mechanizm TTL (Time-To-Live):** Jest to kluczowa zaleta tego rozwiązania. Przy każdym przesłaniu lokalizacji, aplikacja ustawia TTL tego klucza na 60 sekund. Jeśli kurier straci połączenie z internetem (np. wjedzie do tunelu) lub jego aplikacja ulegnie awarii, pingi przestaną docierać. Po minucie Redis bezgłośnie i automatycznie usunie ten klucz.
- **Korzyść biznesowa:** Moduł szukający kurierów opiera się tylko na istniejących sesjach. Eliminujemy w ten sposób ryzyko przypisania zamówienia do "martwego" kuriera bez pisania skomplikowanych skryptów czyszczących (cron jobs) w głównej bazie danych.

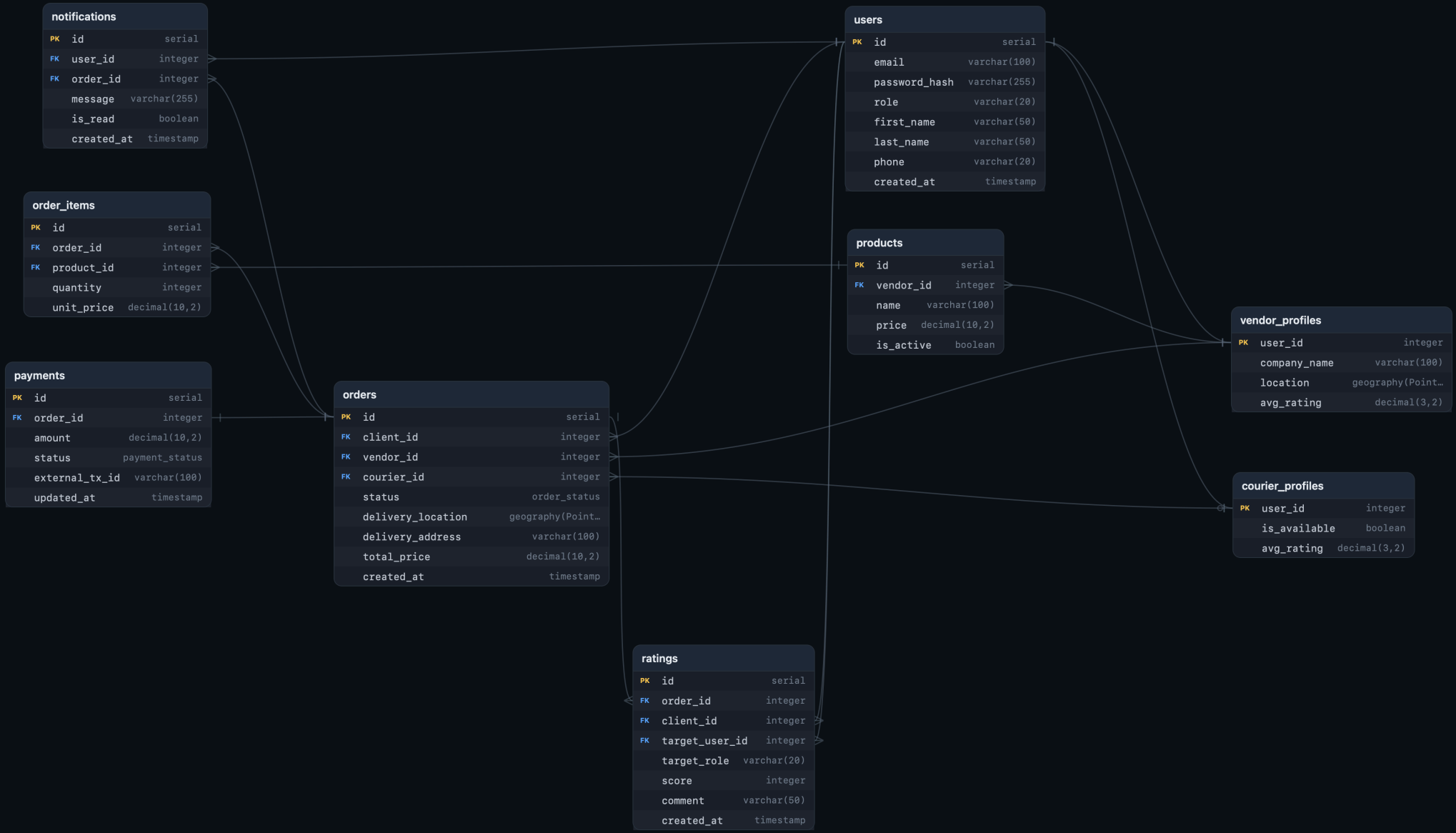
C. order:lock:[order_id] (Rozproszona blokada / STRING)

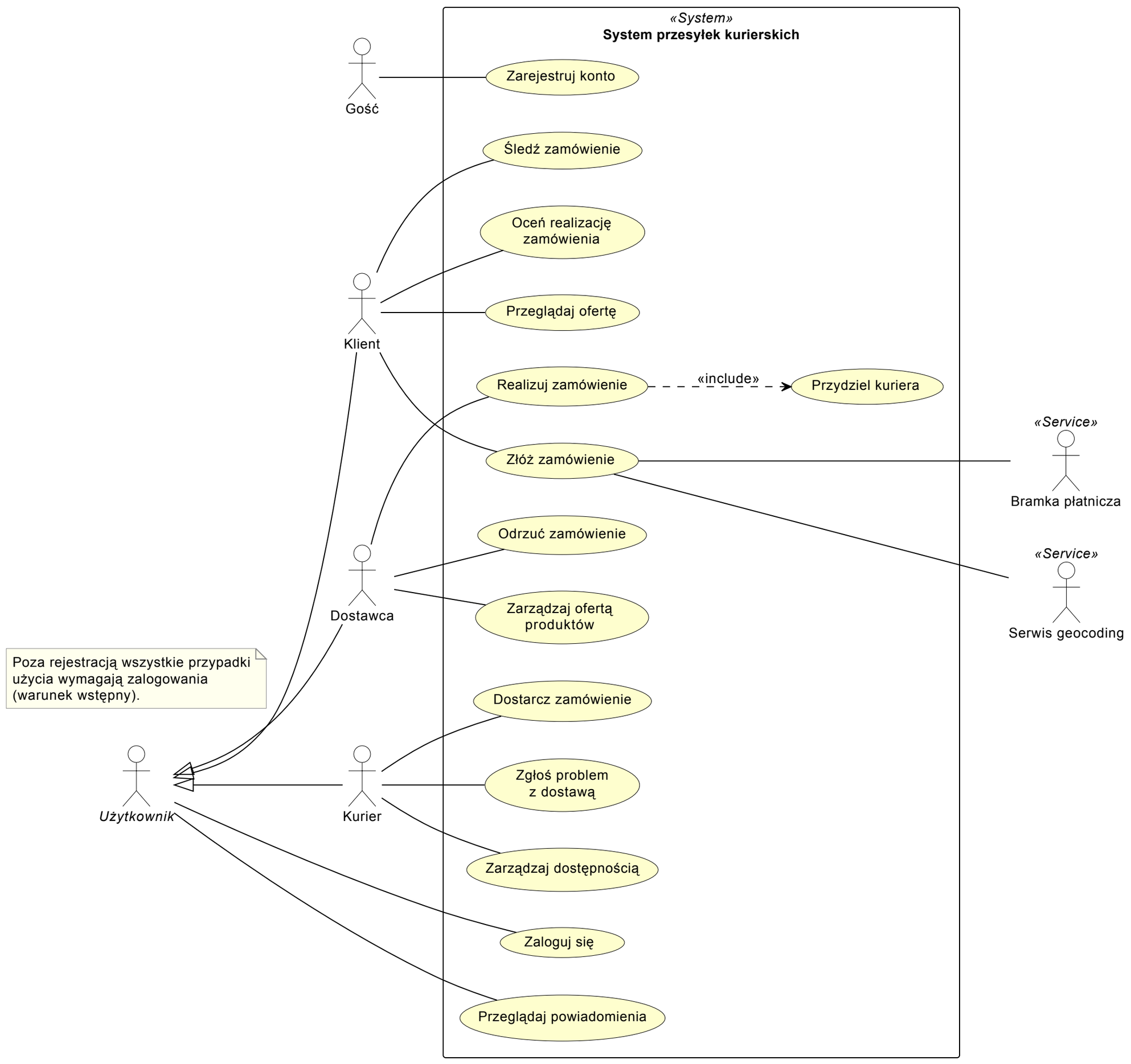
Najprostszy typ danych (ciąg znaków), wykorzystywany do zabezpieczenia automatu stanów przed zjawiskiem wyścigu (race condition).

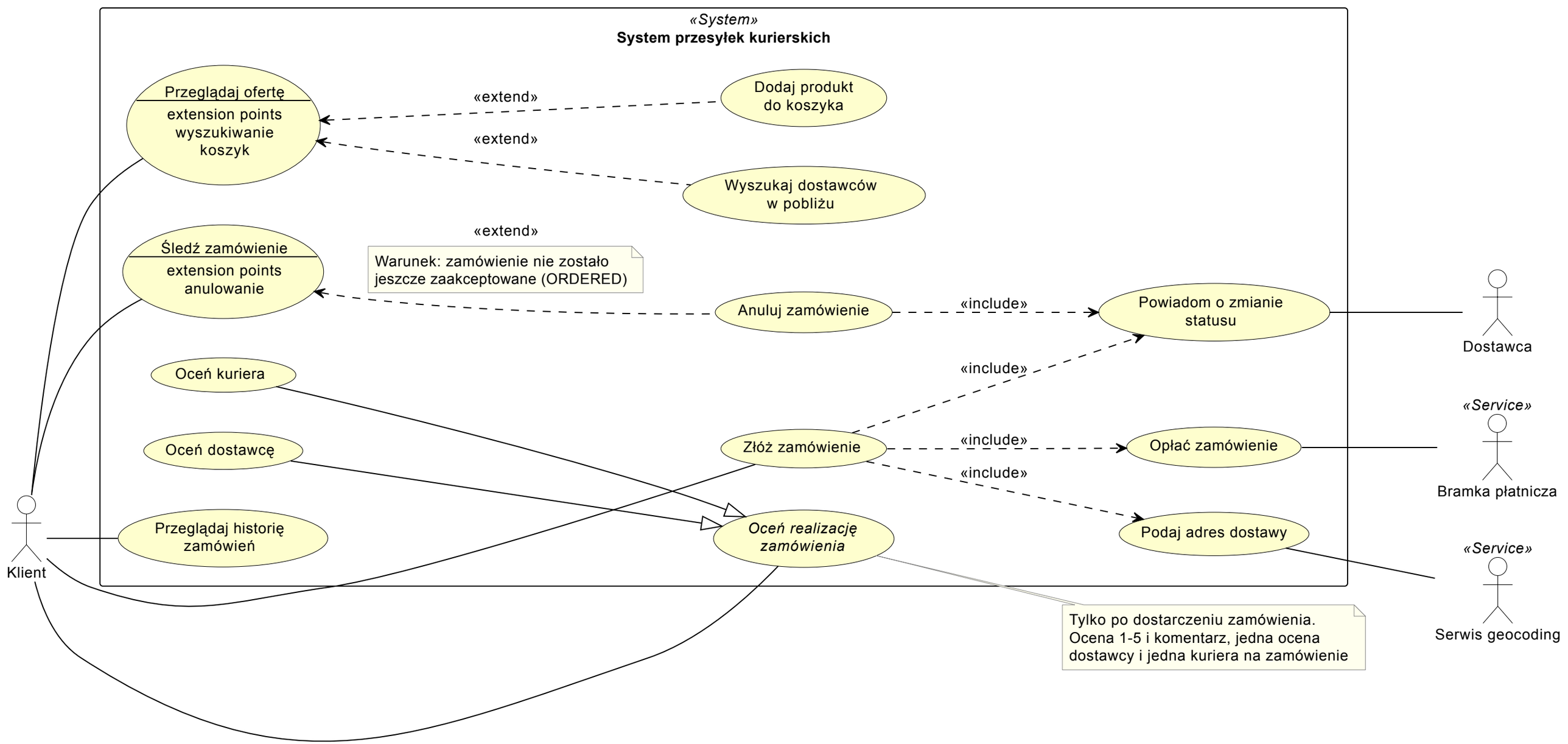
- **Problem:** Dwa niezależne procesy w tym samym ułamku sekundy mogłyby spróbować przypisać kurierów do tego samego zamówienia (np. jeśli klient ponowi żądanie lub nastąpi błąd w komunikacji).
- **Rozwiązanie:** Przed aktualizacją bazy PostgreSQL, system wykonuje w Redisie komendę `SETNX` (Set if Not eXists) dla klucza `order:lock:15`. Baza gwarantuje atomowość tej operacji - zablokuje zamówienie dla pierwszego procesu, a drugi otrzyma odmowę.
- **Zabezpieczenie przed zawieszeniem:** Klucz ten otrzymuje bardzo krótki TTL (np. 3 sekundy). Nawet jeśli serwer ulegnie krytycznej awarii tuż po założeniu blokady i nie zdąży jej zdjąć, zamek samoczynnie zwolni się po 3 sekundach, odblokowując zamówienie.

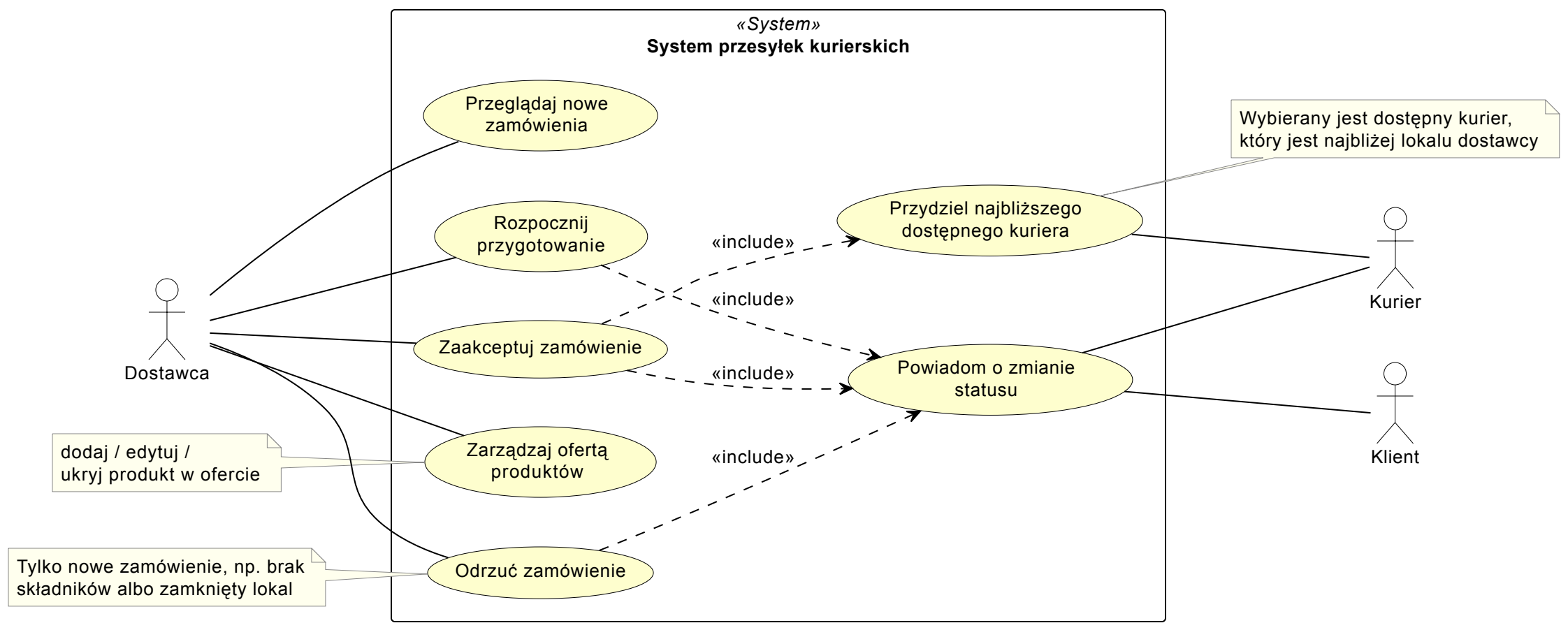

«GEO - Zbiór przestrzenny»
CourierLocations
+String KEY "couriers:locations"
+Float score "Współrzędne GPS"
+String opis "Błyskawiczne wyszukiwanie przez GEOSEARCH"
+String member "np. 42(courier_id) : "
«STRING - SETNX›
OrderAssignLock
+String KEY "order:lock:[order_id]"
+Integer TTL "3 sekundy"
+String value "LOCKED"
+String opis "Rozproszona blokada(Mutex) : "
Zawsze synchronizowane
razem (Ping GPS)
«HASH - Słownik»
CourierSession
+String KEY "courier:session:[courier_id]"
+Boolean is_available
+Timestamp last_ping
+Integer current_order_id
+String opis "Bieżący status, zastępuje UPDATE w SQL"
+Integer TTL "60 sekund(Auto-wygasanie) : "


Warstwa Kliencka
Panel Klienta (React)
Panel Dostawcy (React)
Panel Kuriera (React Native)
HTTP
HTTP
HTTP
Warstwa Serwerowa NestJS
REST API (Kontrolery)
Logika Biznesowa i Serwisy
Autoryzacja i Użytkownicy
Oceny i Opinie
Zamówienia (Automat Stanów)
Zewnętrzny WebSocket
Zewnętrzny WebSocket
Zewnętrzny WebSocket
Walidacja produktów
Wyszukiwanie kuriera
Delegacja płatności
Zlecanie wysyłki alertów
Katalog Produktów
Geolokalizacja i Przydział
Moduł Płatności
Powiadomienia
Telemetria na żywo / Soft-locki
Zapytania ST_DWithin / Dane stałe
Konwersja adresu na współrzędne API
Inicjalizacja płatności API
Asynchroniczny Webhook
Wypychanie powiadomień API
Warstwa Składowania Danych
Redis (Baza In-memory)
PostgreSQL + PostGIS (Baza główna)
Usługi Zewnętrzne
Nominatim OSM (Geokodowanie)
Stripe (Bramka Płatnicza)
Ably Platform (Real-Time Pub/Sub)


notifications
PK id serial
FK user_id integer
FK order_id integer
message varchar(255)
is_read boolean
created_at timestamp
users
PK id serial
email varchar(100)
password_hash varchar(255)
role varchar(20)
first_name varchar(50)
last_name varchar(50)
phone varchar(20)
created_at timestamp
order_items
PK id serial
FK order_id integer
FK product_id integer
quantity integer
unit_price decimal(10,2)
products
PK id serial
FK vendor_id integer
name varchar(100)
price decimal(10,2)
is_active boolean
vendor_profiles
PK user_id integer
company_name varchar(100)
location geography(Point…
avg_rating decimal(3,2)
payments
PK id serial
FK order_id integer
amount decimal(10,2)
status payment_status
external_tx_id varchar(100)
updated_at timestamp
orders
PK id serial
FK client_id integer
FK vendor_id integer
FK courier_id integer
status order_status
delivery_location geography(Point…
delivery_address varchar(100)
total_price decimal(10,2)
created_at timestamp
courier_profiles
PK user_id integer
is_available boolean
avg_rating decimal(3,2)
ratings
PK id serial
FK order_id integer
FK client_id integer
FK target_user_id integer
target_role varchar(20)
score integer
comment varchar(50)
created_at timestamp


«System»
System przesyłek kurierskich
Gość
Zarejestruj konto
Śledź zamówienie
Oceń realizację zamówienia
Klient
Przeglądaj ofertę
Realizuj zamówienie
«include»
Przydziel kuriera
«Service»
Złóż zamówienie
Bramka płatnicza
Odrzuć zamówienie
Dostawca
«Service»
Zarządzaj ofertą produktów
Serwis geocoding
Poza rejestracją wszystkie przypadki użycia wymagają zalogowania (warunek wstępny).
Dostarcz zamówienie
Użytkownik
Kurier
Zgłoś problem z dostawą
Zarządzaj dostępnością
Zaloguj się
Przeglądaj powiadomienia


«System»
System przesyłek kurierskich
Przeglądaj ofertę
extension points
wyszukiwanie
koszyk
Dodaj produkt
do koszyka
«extend»
«extend»
Wyszukaj dostawców
w pobliżu
Śledź zamówienie
extension points
anulowanie
«extend»
Warunek: zamówienie nie zostało
jeszcze zaakceptowane (ORDERED)
Anuluj zamówienie
«include»
Powiadom o zmianie
statusu
Dostawca
Oceń kuriera
Oceń dostawcę
Przeglądaj historię
zamówień
Klient
Złóż zamówienie
«include»
«include»
«include»
Opłać zamówienie
Podaj adres dostawy
«Service»
Bramka płatnicza
«Service»
Serwis geocoding
Oceń realizację
zamówienia
Tylko po dostarczeniu zamówienia.
Ocena 1-5 i komentarz, jedna ocena
dostawcy i jedna kuriera na zamówienie


«System»
System przesyłek kurierskich
Przeglądaj nowe zamówienia
Rozpocznij przygotowanie
Zaakceptuj zamówienie
Zarządzaj ofertą produktów
Odrzuć zamówienie
Przydziel najbliższego dostępnego kuriera
Powiadom o zmianie statusu
«include»
«include»
«include»
«include»
Dostawca
Kurier
Klient
Wybierany jest dostępny kurier, który jest najbliżej lokalu dostawcy
dodaj / edytuj / ukryj produkt w ofercie
Tylko nowe zamówienie, np. brak składników albo zamknięty lokal

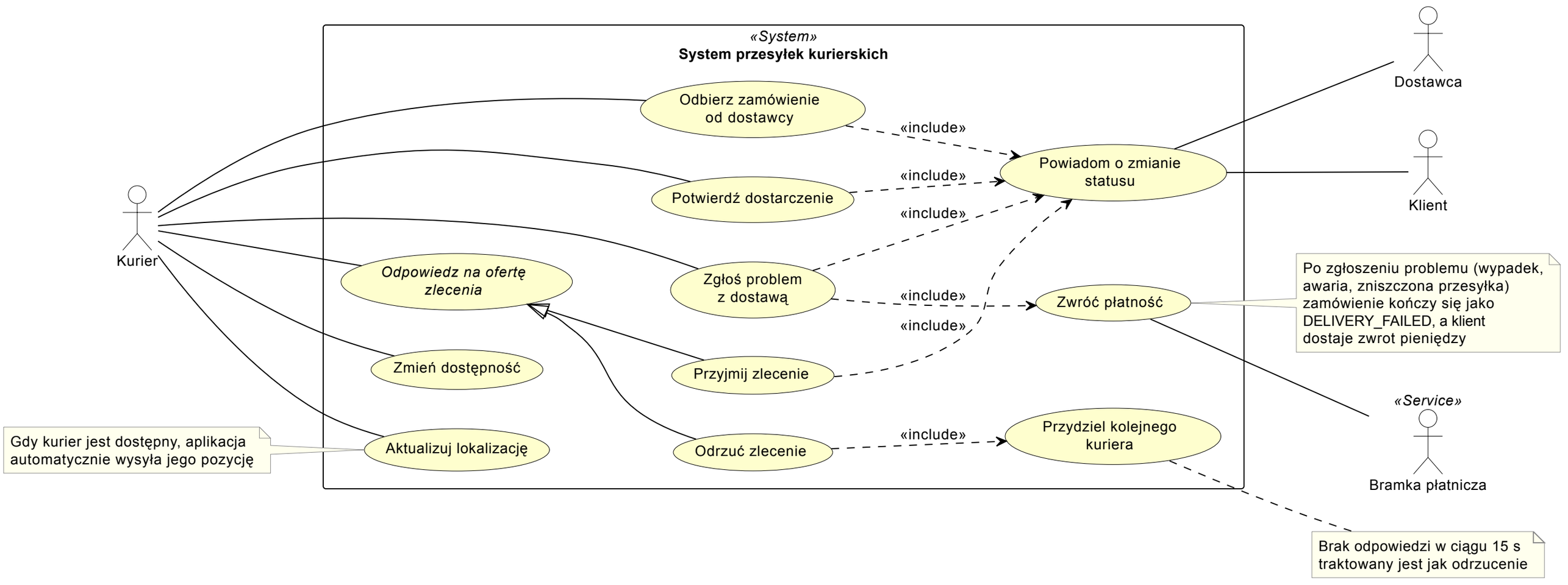

«System»
System przesyłek kurierskich
Odbierz zamówienie od dostawcy
Potwierdź dostarczenie
Odpowiedz na ofertę zlecenia
Zgłoś problem z dostawą
Zmień dostępność
Przyjmij zlecenie
Aktualizuj lokalizację
Odrzuć zlecenie
Powiadom o zmianie statusu
Zwróć płatność
Przydziel kolejnego kuriera
«include»
«include»
«include»
«include»
«include»
«include»
Kurier
Dostawca
Klient
«Service»
Bramka płatnicza
Gdy kurier jest dostępny, aplikacja automatycznie wysyła jego pozycję
Po zgłoszeniu problemu (wypadek, awaria, zniszczona przesyłka) zamówienie kończy się jako DELIVERY_FAILED, a klient dostaje zwrot pieniędzy
Brak odpowiedzi w ciągu 15 s traktowany jest jak odrzucenie

Diagram sekwencji 1: Rejestracja i logowanie

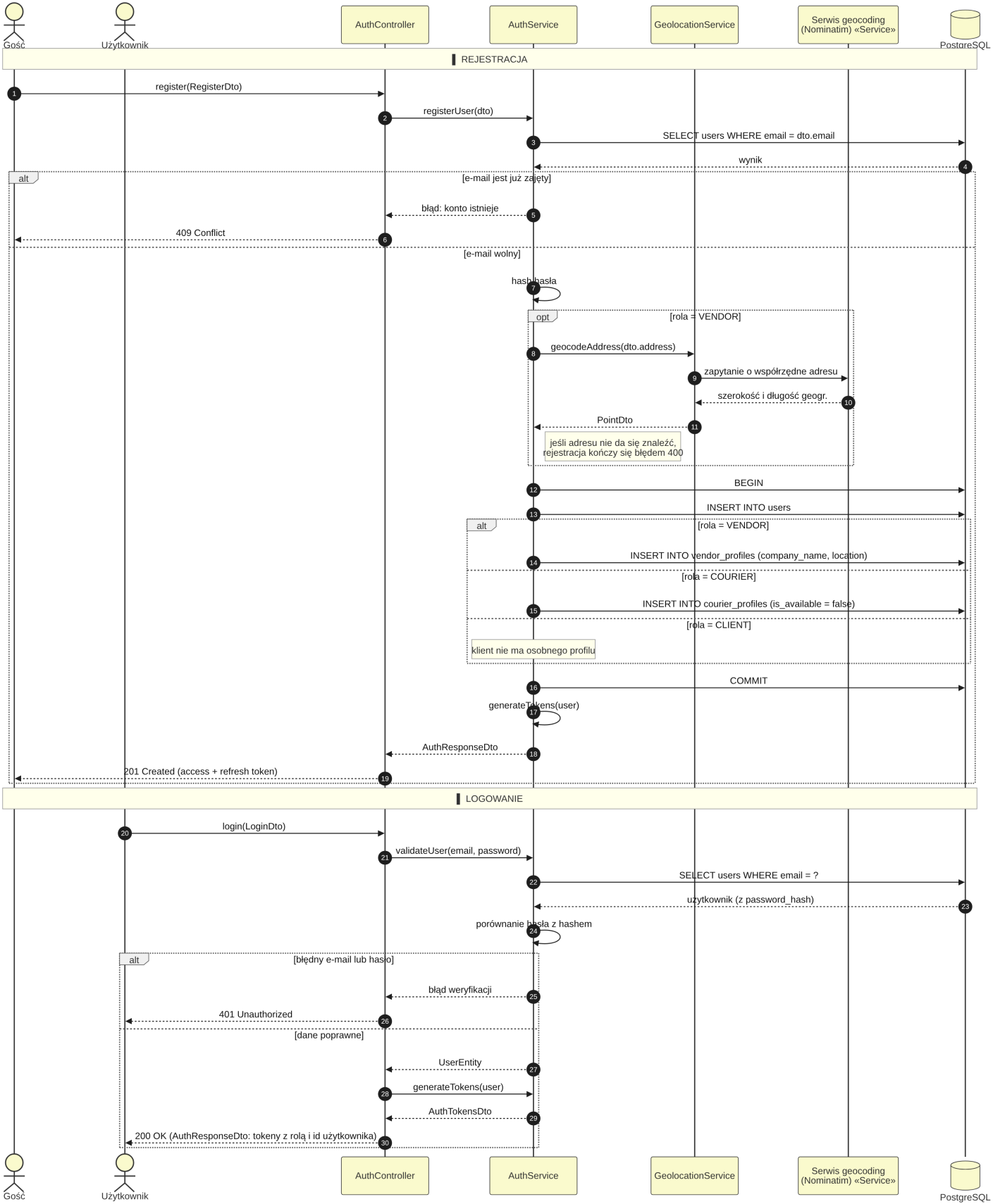


Diagram sekwencji 2: Złożenie i opłacenie zamówienia

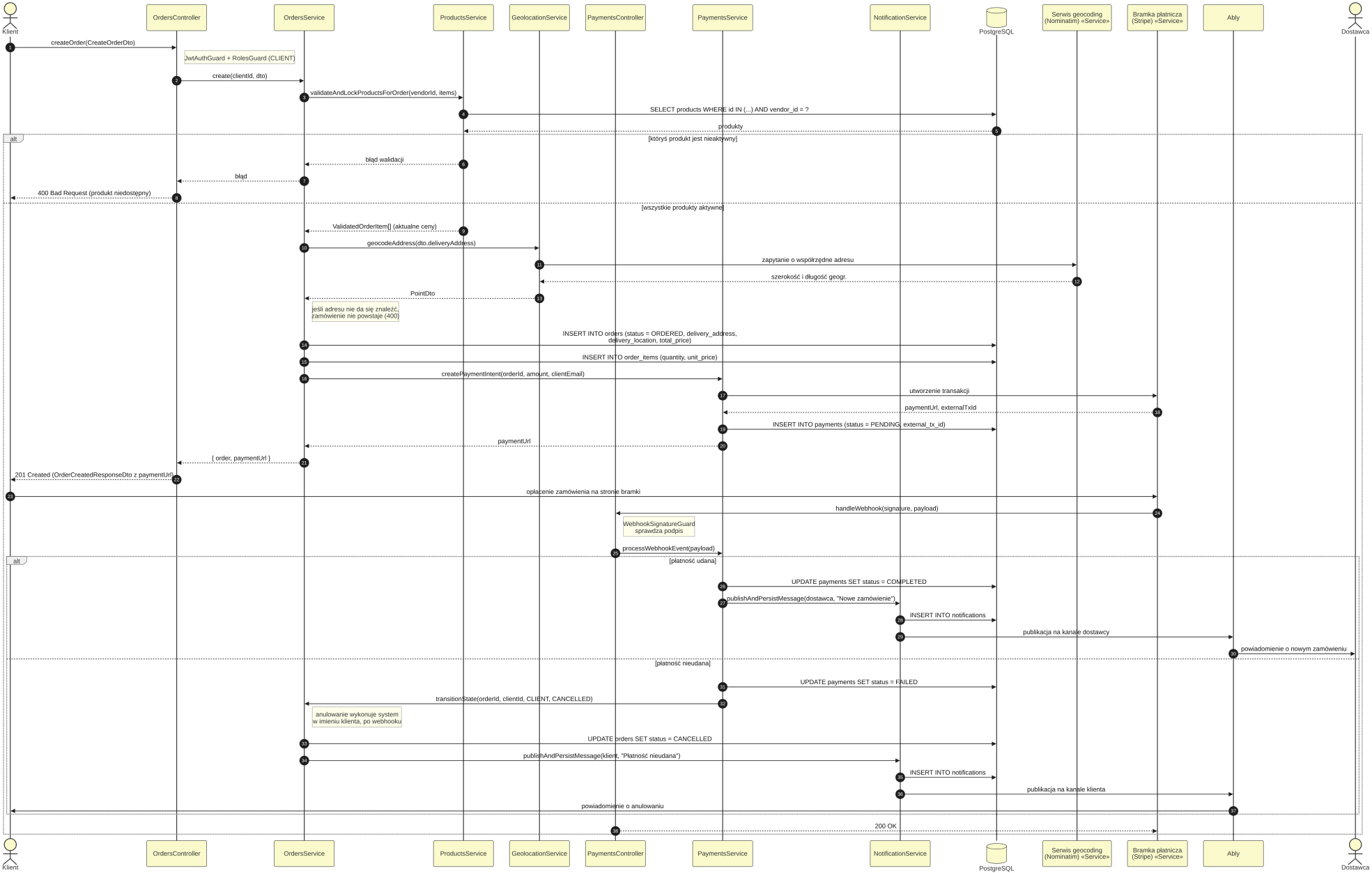


Diagram sekwencji 3: Akceptacja zamówienia i przydział kuriera

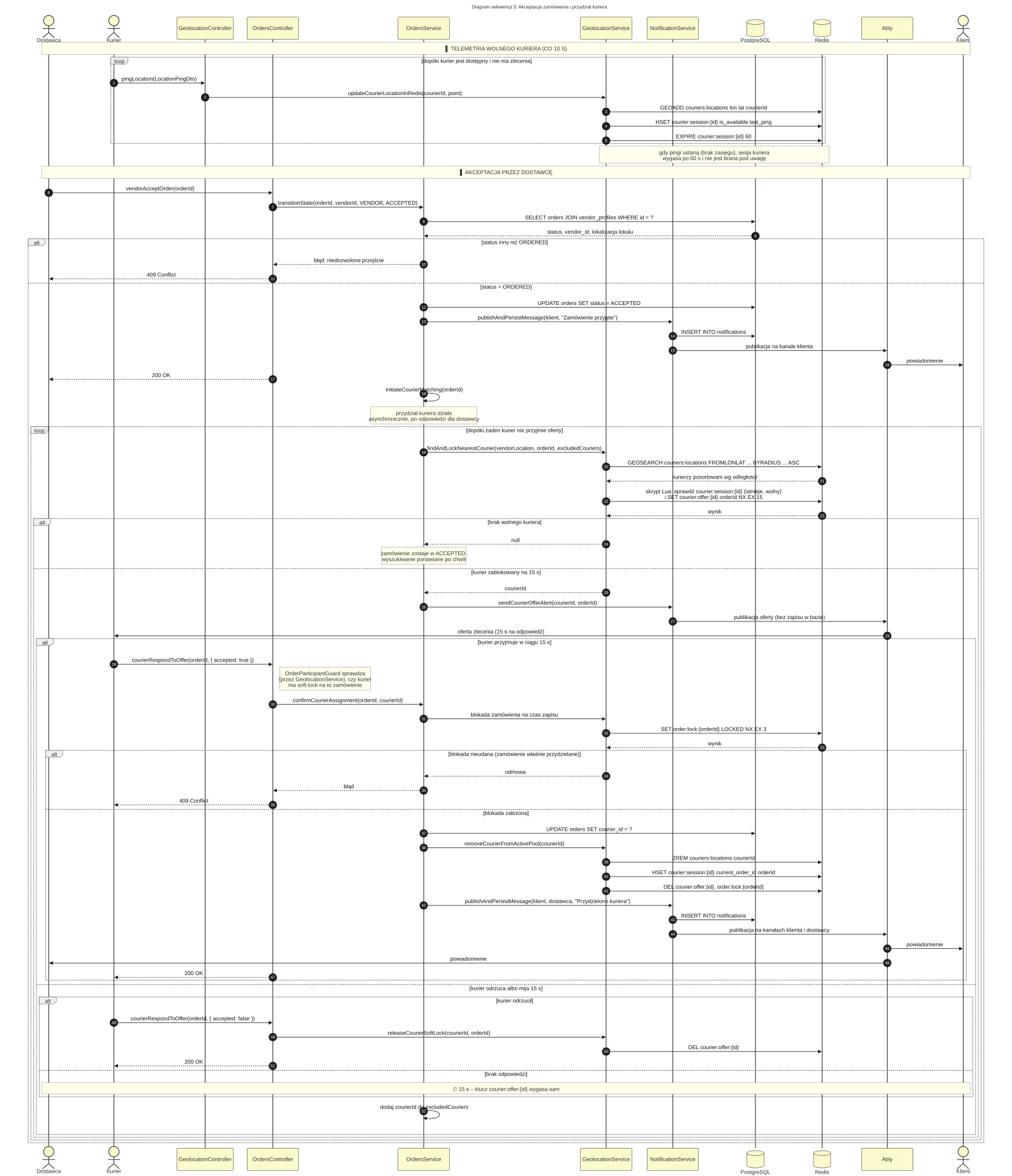


Diagram sekwencji 4: Odrzucenie zamówienia przez dostawcę

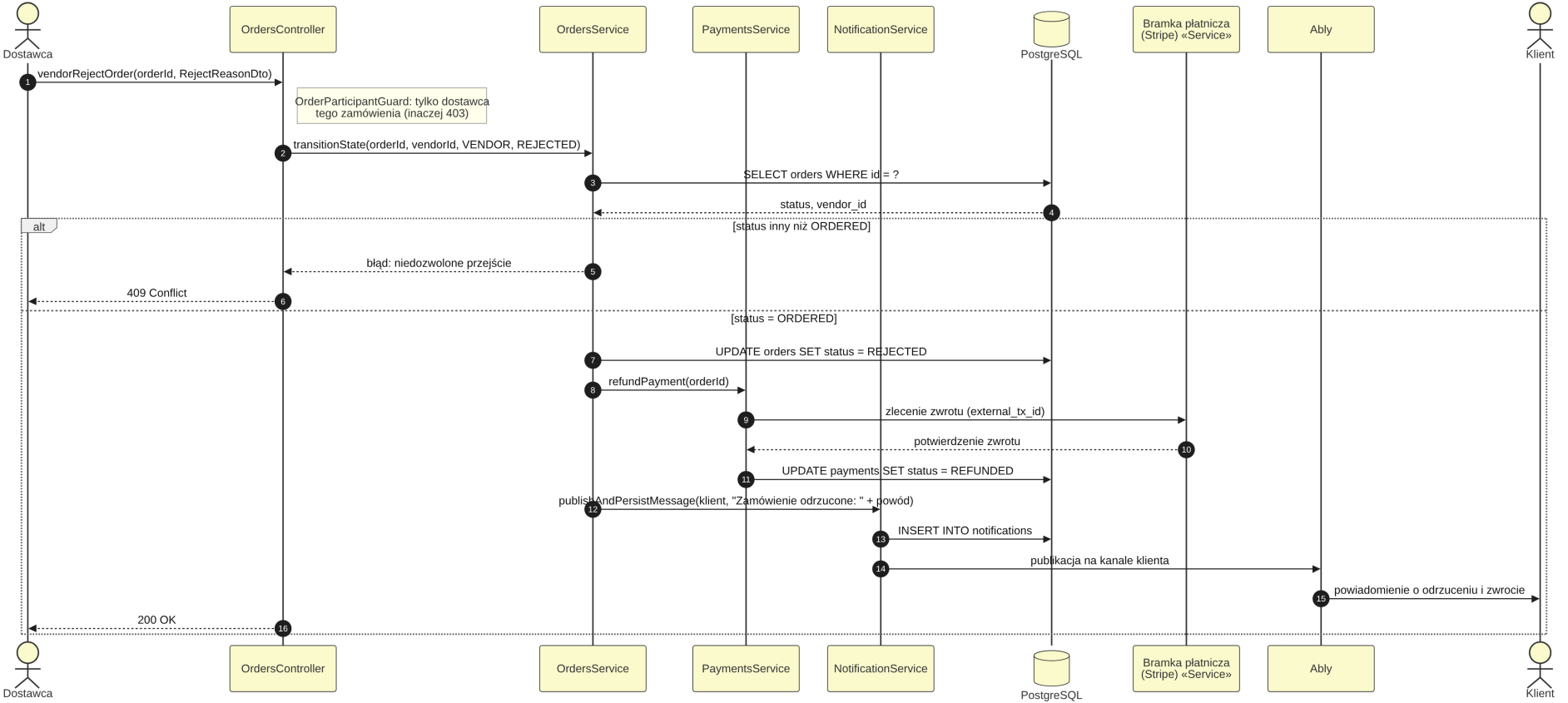


Diagram sekwencji 5: Przygotowanie, odbiór, dostawa i śledzenie

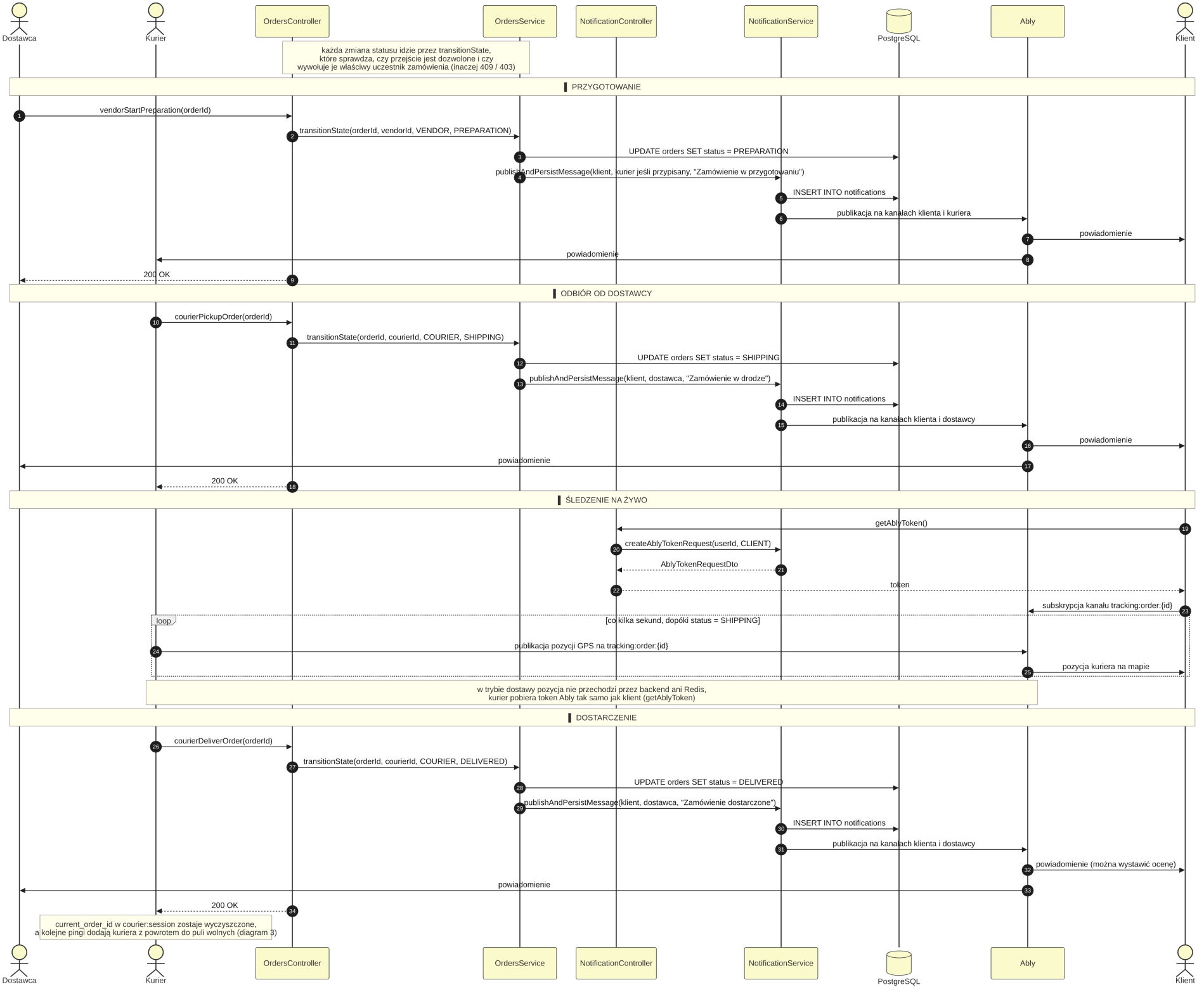


Diagram sekwencji 6: Zgłoszenie problemu z dostawą

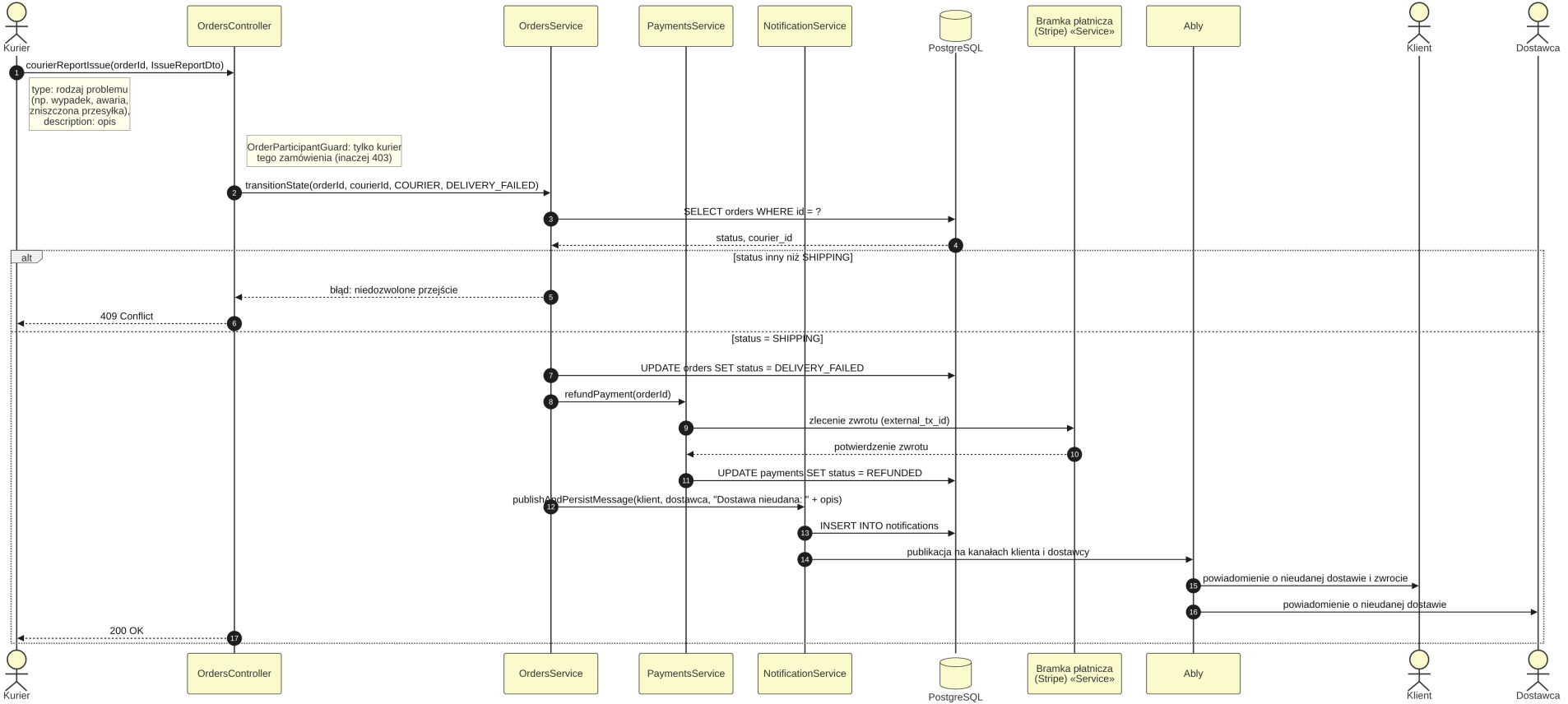


Diagram sekwencji 7: Anulowanie zamówienia przez klienta

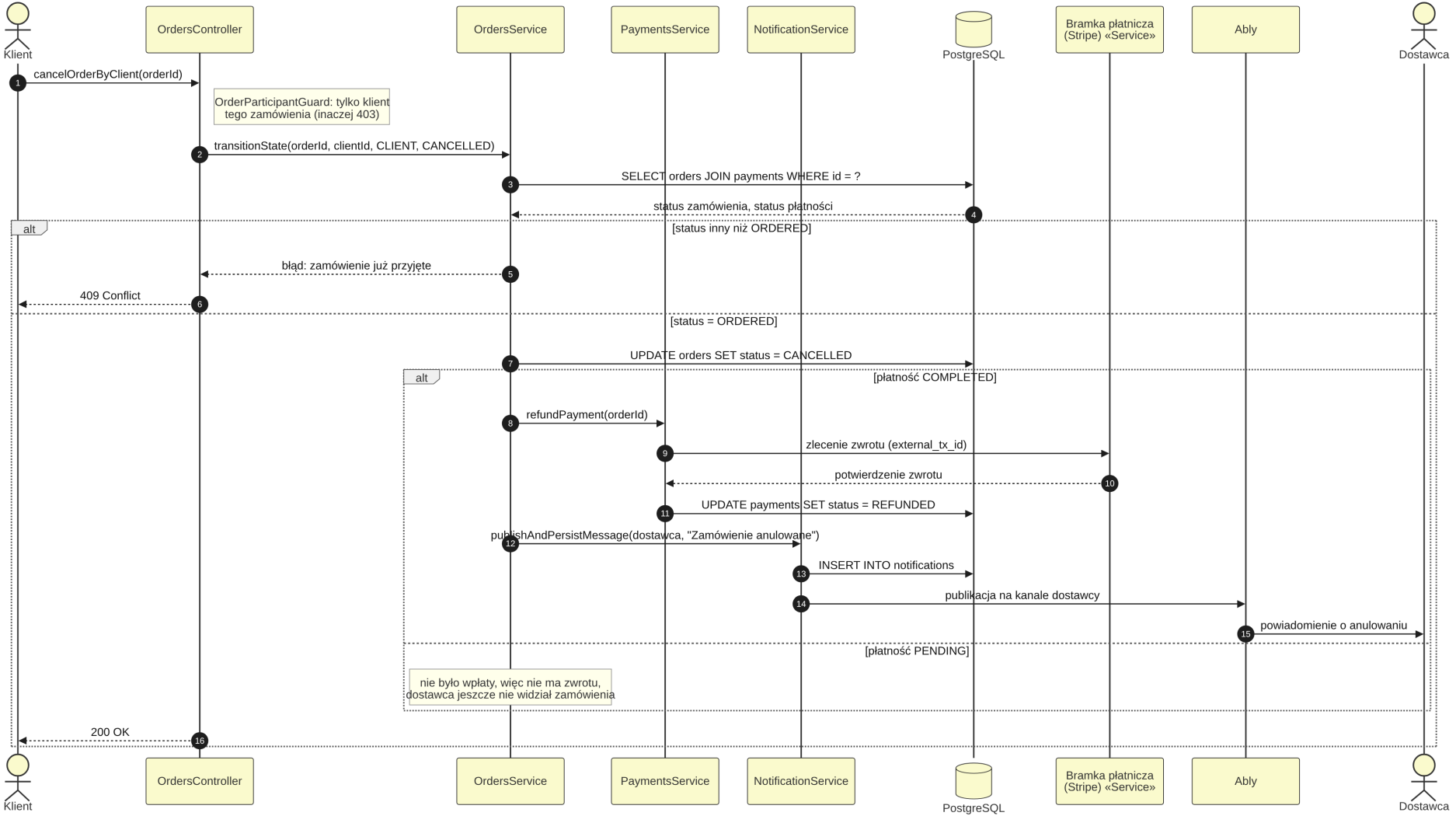


Diagram sekwencji 8: Wystawienie oceny

